Codeit AI Sprint 15기 · 초급 프로젝트 (개인)

경구약제 이미지 객체 검출

사진 한 장에서 최대 4개 알약의 종류(56종)와 위치를 찾는다

기간	2026.10.27 – 2026.11.05
작성	[이름] (github.com/lee-da-gyo)
저장소	github.com/lee-da-gyo/ai-project-01-pill-detection
환경	Python 3.14 · PyTorch 2.11 · Ultralytics 8.4 · RTX 3060 12GB

요약

학습 이미지는 232장, 약은 56종이며 절반 이상이 10개 미만의 박스만 가진 롱테일 데이터다. 같은 알약 조합을 각도만 바꿔 최대 세 번(조합당 1~3장) 찍은 구조라, 이미지 단위로 나누면 검증 점수가 부풀려진다. 이 프로젝트는 **모델을 바꾸기 전에 "정직하게 재는 법"을 먼저 만들었다.**

10건	**0건**	**0.954**	**[Kaggle]**
라벨 오류 발견·수정 (라벨과 독립적인 배경 분할로 감사)	같은 조합의 검증 누수 (조합 단위 5-Fold)	현재 최종 후보의 5-Fold OOF mAP50-95 (37종 조건부 진단값, 단일 모델 0.941)	최종 제출 public 점수 (제출 후 기입)

- **핵심 발견** – 오류는 위치가 아니라 **학습 때 본 적 없는 약의 이름**에서 나온다 (학습한 약 AP 0.985, 처음 보는 약 0.000).
- **현재 최종 후보** – YOLO11s + YOLO11m을 232장 전체로 학습하고 WBF(1:1, IoU 0.7)로 합친다. 설정은 5-Fold 예측으로 골랐고, fold를 하나씩 보류하는 근사 메타 검증 5회에서 모두 같은 설정이 뽑혔다.
- **기각한 시도** – 고해상도, YOLO26, Copy-Paste 합성, 2단계(검출+분류기), CLAHE 전처리. 각 시도의 기각 근거를 수치로 남겼다.
- **한계** – 한 fold에만 있는 희귀 19종은 검증 때마다 학습에 없어, 이 데이터로는 그 약들의 실제 성능을 잴 수 없다.

목차

1. 문제 정의
2. 데이터 분석 (EDA)
3. 데이터 정제
4. 검증 전략
5. 모델과 실험 설계
6. 실험 결과와 의사결정
7. 최종 후보 모델과 제출
8. 결과 분석
9. 재현성과 코드 품질
10. 업무 수행 과정
11. 비즈니스 시사점과 한계
12. 회고와 향후 과제

1. 문제 정의

1.1 배경

헬스케어 스타트업 **헬스잇(Health Eat)**은 사용자가 복용 중인 약을 촬영하면 어떤 약인지 자동으로 알려 주는 기능을 만들고자 한다. 약을 정확히 알아보면 **함께 먹으면 안 되는 약 안내**, 복약 기록 같은 서비스로 확장할 수 있다. 이 프로젝트는 그 첫 단계인 인식 모델을 만든다.

1.2 과제

항목	내용
과제 유형	객체 검출 (Object Detection): 알약마다 약 종류(클래스)와 바운딩 박스를 함께 예측
입력	단색 배경 위에 알약 3~4개가 놓인 사진 (976 × 1280)
데이터	학습 232장 (박스 771개, 56종) · 테스트 842장 (정답 비공개, Kaggle로 채점)
제약	원본 공개 데이터(AI Hub 경구약제 이미지 데이터[12]의 조합 데이터) 사용 금지 · Kaggle 하루 10회 제출

1.3 평가 지표

주 지표는 **mAP@[.5:.95]**(이하 mAP50-95)다. IoU 0.5부터 0.95까지 10개 기준에서 AP를 평균하므로, 약 이름을 맞혔는지와 함께 **박스를 얼마나 정확히 그렸는지**까지 평가한다. 검증에는 pycocotools[3]로 직접 계산한 COCO mAP[2]를 쓴다. 베이스라인에서 Ultralytics 내장 평가(0.834)와 직접 구현(0.835)이 일치함을 확인했고, 덕분에 앙상블·2단계 모델처럼 Ultralytics 밖에서 만든 예측도 같은 기준으로 비교할 수 있었다.

> Kaggle 대회 페이지의 평가 지표와 제출 형식 확인 결과를 여기에 기입한다 (제출 형식 확인 후).

2. 데이터 분석 (EDA)

EDA의 목적은 그래프를 많이 그리는 것이 아니라 **데이터의 특성을 설계 결정으로 바꾸는 것**이다. 발견 6가지를 각각 하나의 결정으로 연결했다.

발견	근거	결정
데이터가 매우 적다	학습 232장으로 테스트 842장을 예측	COCO 사전학습 YOLO 파인튜닝
심한 클래스 불균형	최대 157개 vs 최소 3개 (약 50배). 56종 중 29종이 박스 10개 미만	클래스별 AP 추적, 희귀 약 보강 시도 (Copy-Paste)
사실상 1장짜리 클래스	17종은 같은 장면을 각도만 바꾼 3장이 전부	증강으로 맥락 다양화, 성능 측정의 한계로 명시
같은 조합을 여러 각도로	114개 조합을 최대 세 각도(70° / 75° / 90°)로 촬영. 실제 보유 수는 3장 40조합, 2장 38조합, 1장 36조합	2~3장인 78개 조합은 거의 같은 장면이 반복된다 → 이미지 단위 분할은 누수 → **조합 단위 분할** (4장)
고정된 배치 위치	박스 중심이 몇 개 슬롯에 몰려 있음	이동·모자이크 증강으로 위치 편향 방지
Train/Test 분포 차이	Test가 약간 어둡고(밝기 131.5 → 125.9) 대비가 큼(23.6 → 26.5)	밝기·대비 증강, CLAHE 전처리 비교 (6.5절)

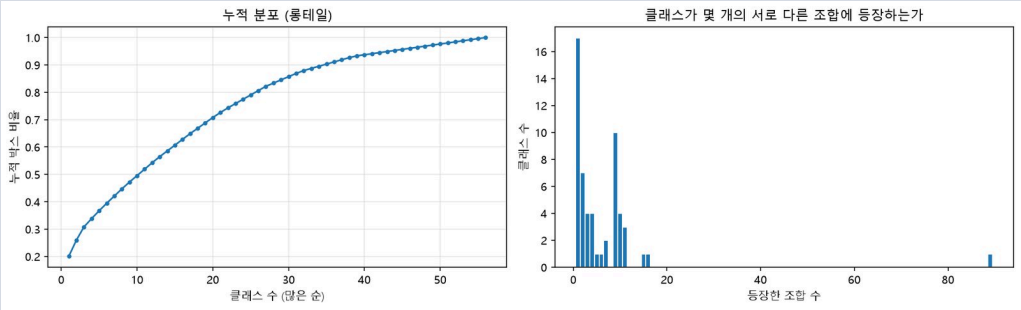


그림 2-1. 왼쪽: 상위 10종이 박스의 49.5%를 차지하는 롱테일. 오른쪽: 17종은 단 하나의 조합에만 등장한다



그림 2-2. 같은 조합을 각도만 바꿔 촬영 → 이미지 단위로 나누면 누수

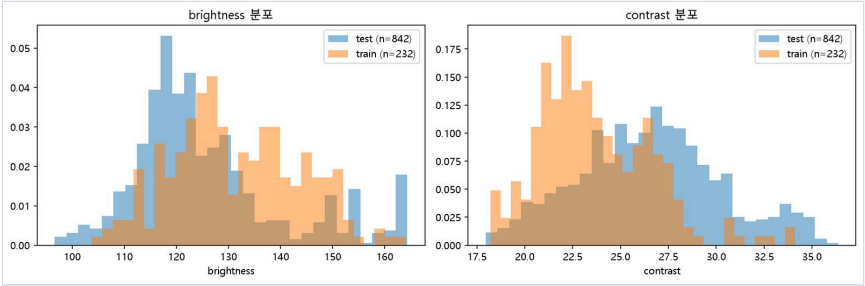


그림 2-3. Train vs Test 밝기·대비 분포 (244×320 썸네일의 RGB 평균·표준편차)

3. 데이터 정제

라벨이 틀렸는지 확인하려면 **라벨과 독립적인 기준**이 필요하다. 배경이 단색에 가깝다는 EDA 결과를 이용해, 배경 차분(Lab 색 거리 + Otsu 이진화)으로 알약 영역을 분할하고 라벨과 대조했다. 또 하나의 기준은 파일명이다. 파일명에는 조합의 약 코드가 들어 있고, 232장 중 224장에서 라벨의 약 목록과 정확히 일치했다 (나머지 8장은 모두 "라벨 하나가 빠진" 경우).

오류 유형	탐지 규칙	수정	건수
범위 밖 박스	좌표가 이미지 밖 (x = 6567)	숫자 하나를 뺀 후보 중 라벨 없는 알약과 가장 잘 맞는 값 → x = 567	1
엉뚱한 위치	박스 안이 배경 + 겹치지 않는 같은 크기의 라벨 없는 알약 존재	그 알약 위치로 이동	1
누락 박스	파일명의 약 코드 중 라벨 없는 것 1개 + 라벨 없는 알약 1개	해당 코드로 박스 추가 (같은 조합 다른 각도 사진과 생김새 대조)	8

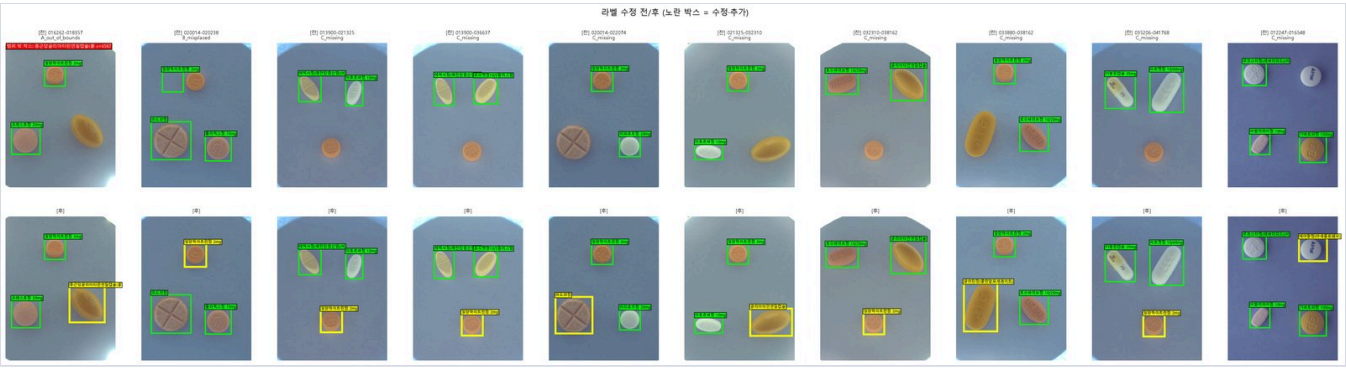


그림 3-1. 라벨 수정 전후 (위: 원본, 아래: 정제. 노란 박스 = 수정·추가)

> **실패에서 배운 점** – 처음 만든 "빈 박스 이동" 규칙은 6건을 고쳤지만 그중 **5건이 오탐**이었다. 반투명 캡슐과 파란 알약은 분할이 잘 되지 않아 박스가 비어 보였던 것이다. "원래 박스와 겹치지 않고 크기가 비슷한 알약이 따로 있다"는 조건을 더해 진짜 오류 1건만 남겼다. 자동 규칙의 결과 10건은 모두 이미지로 육안 검수했다 (`docs/LABEL_REVIEW.md`).

최종 무결성 검사 (코드로 자동 확인): 모든 이미지에서 파일명의 약 목록 = 라벨의 약 목록 · 범위 밖·크기 0·중복 박스 0 · YOLO 라벨 → 픽셀 좌표 왕복 오차 < 0.001px.

4. 검증 전략

같은 조합의 다른 각도 사진이 학습과 검증에 나뉘어 들어가면, 모델은 거의 같은 장면을 다시 맞히는 셈이 되어 점수가 부풀려진다. 이 프로젝트에서 가장 중요한 설계는 모델이 아니라 이 누수를 막는 분할이었다.

- scikit-learn의 StratifiedGroupKFold[10]를 썼다.
- **Group = 조합**: 한 조합의 사진(1~3장)을 반드시 같은 fold에 넣는다 (누수 0건을 코드로 검사).
- **Stratify = 이미지의 가장 희귀한 클래스**: 희귀 약이 fold에 고르게 퍼지도록 한다.
- 5-Fold는 실험 비교에만 쓰고, 최종 제출 모델은 232장 **전체**로 학습한다 (7장).

Fold	검증 이미지	검증 조합	검증 박스	학습에 없는 약	베이스라인 mAP50-95
0	47	23	156	5	0.835
1	46	22	153	3	0.889
2	46	23	150	1	0.920
3	46	22	153	7	0.740
4	47	24	159	3	0.881

표 4-1. fold 구성과 베이스라인(YOLO11s) 점수. 처음 보는 약 수와 점수 순위가 정확히 반대다.

4.1 open-set 상한과 비교 지표

조합 단위로 나누면 한 조합에만 있는 약은 검증될 때 반드시 학습에 없다. 56종 중 **19종이 한 fold에만** 있어, 5개 fold의 검증 예측을 합친 OOF(out-of-fold) mAP의 상한은 37/56 = **0.661**이다. 따라서 56종 OOF는 모델보다 분할 구조에 좌우된다.

> **비교 지표의 결정** – 모델·앙상블은 2개 이상 fold에 있는 **37종만으로 다시 잰 OOF**(이하 37종 조건부 OOF)로 비교한다. 어려운 희귀 19종을 정답·예측 양쪽에서 뺀 값이므로 **같은 조건에서 모델을 비교하는 진단값**이며, 56종 Kaggle 점수의 추정치가 아니다. 56종을 모두 학습하는 최종 모델에서 희귀 19종이 실제로 얼마나 맞을지는 이 데이터로는 잴 수 없다.

fold 하나(독립 조합 23개)로는 0.005 이하의 차이를 구분할 수 없다. 그래서 fold 0에서 빠르게 탐색하고, 최종 후보는 5-Fold로 다시 비교하는 2단계 절차를 썼다.

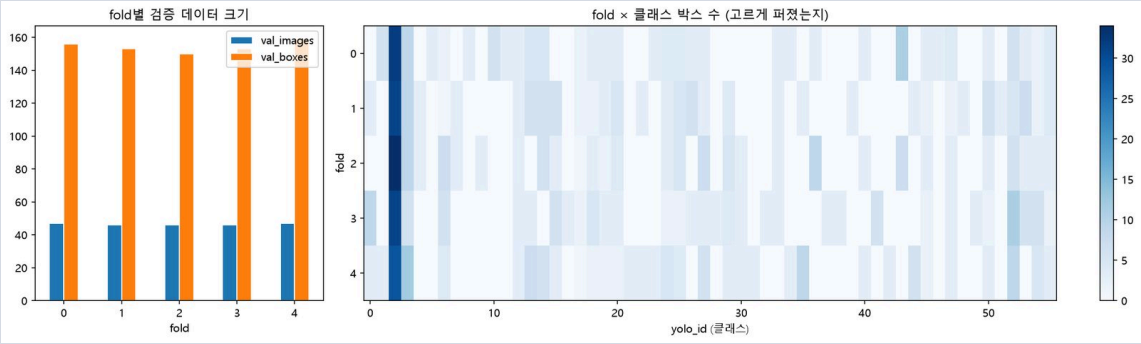


그림 4-1. fold별 클래스 분포

5. 모델과 실험 설계

5.1 모델 선택

데이터가 232장뿐이라 처음부터 학습하는 모델은 맞지 않는다. COCO[2]로 사전학습된 **YOLO11**(Ultralytics)[1]을 파인튜닝했다. 한 단계로 위치와 클래스를 함께 예측해 빠르고, 크기(n / s / m)를 바꿔 용량을 비교하기 쉽다. 약 이름만 따로 판별하는 2단계 구조를 위해서는 분류기 3종(VGG16[7] · ResNet50[8] · EfficientNet-B0[9])을 같은 조건에서 비교했다 (6.3절).

5.2 실험 원칙

- **한 번에 한 가지만 바꾼다.** 모든 실험은 베이스라인 설정 파일을 복사해 값 하나만 바꾼다. 점수가 변하면 원인을 설명할 수 있다.
- **실험 하나 = 설정 파일 하나.** `python scripts/run_experiment.py --config configs/<실험>.yaml` 한 줄로 학습 → 평가 → 테스트 예측 → 제출 파일 검사 → 실험 기록까지 실행된다.
- **검증 데이터로 고른 값은 그 데이터로 평가하지 않는다.** 앙상블 설정·전처리 설정처럼 고를 것이 있으면 선택 편향을 따로 점검한다.

5.3 베이스라인 (E0)

YOLO11s · 640px · Ultralytics 기본 증강 · 100 에포크(조기 종료 30) · fold 0. mAP50-95 **0.835**, mAP50 0.847, 학습 5.7분.

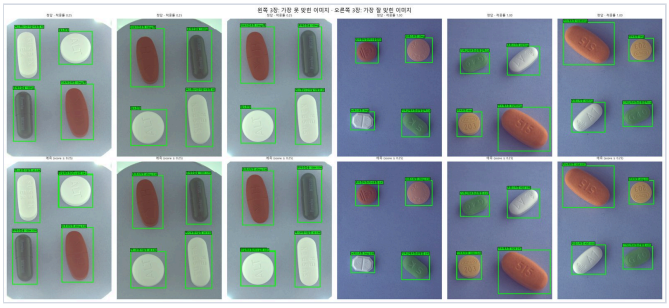


그림 5-1. 위: 정답, 아래: 예측. 위치는 모두 맞고, 처음 보는 약을 비슷한 약으로 예측

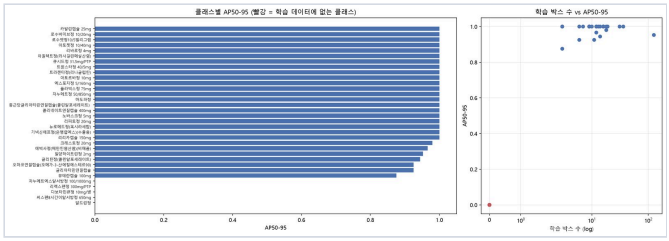


그림 5-2. 클래스별 AP (빨강 = 학습에 없는 약)

발견	근거	다음 결정
오류는 **위치가 아니라 처음 보는 약의 이름**	학습한 약 AP 0.985 / 처음 보는 약 5종 AP 0.000	위치와 이름을 나눠 푸는 2단계 모델 시도
처음 보는 약을 **비슷한 약으로 확신**	다보타민큐정 → 트루비타정 (신뢰도 0.96)	희귀 약 보강 (Copy-Paste), 서비스 설계 시사점 (11장)
이미지당 상위 4개 제한은 **손해**	mAP50-95 0.835 → 0.822	가설 기각. 제출에는 모든 예측 유지

6. 실험 결과와 의사결정

6.1 fold 0 탐색: 해상도 · 모델 크기 · 증강

ID	변경 (E0 대비 하나만)	mAP50	mAP50-95	학습 시간	판단
E0	베이스라인 YOLO11s · 640px	0.847	0.835	5.7분	기준
E1	해상도 960px	0.824	0.811	11.6분	하락
E1	해상도 1280px	0.837	0.830	41.3분	개선 없음, 7배 느림
E2	YOLO11n	0.842	0.823	6.3분	하락
E2	**YOLO11m**	**0.847**	**0.839**	**30.8분***	**최고, 5-Fold 후보**
E2	YOLO26s (최신 구조)	0.828	0.810	4.3분	하락
E3	상하 뒤집기 증강	0.847	0.833	5.3분	차이 없음
E3	밝기 증강 강화 (hsv_v 0.4 → 0.6)	0.847	0.837	5.6분	+0.002, 판단 불가

표 6-1. fold 0(47장) 결과. * 다른 학습과 GPU를 함께 쓴 구간이 있어 시간이 부풀려짐 (5-Fold에서는 fold당 8~11분).

강한 후보들의 mAP50이 0.847로 같다. fold 0 검증에는 학습에 없는 약이 5종 있어 이 분할에서 가능한 mAP50의 최댓값이 28/33 = **0.848**이기 때문이다. 즉 fold 0에서는 **이미 상한에 닿아** 모델 간 차이가 드러나지 않는다. 남은 여지는 박스 정밀도(mAP50-95)와 데이터 전체 활용이므로, 다음 비교는 5-Fold로 옮겼다.

6.2 처음 보는 약 문제: 2단계 모델

오류가 "이름"에서 나오므로 위치(알약 찾기)와 이름(약 판별)을 나눠, YOLO가 찾은 박스를 별도 분류기로 다시 판별하는 2단계 구조를 시도했다.

6.3 분류기 3종 비교 (5-Fold)

2단계 모델의 약 이름 판별기 후보를 **같은 증강·학습 설정에서 구조만 바꿔** 비교했다. 정답 박스를 잘라 학습한 약만으로 평가한 값이라 실제 검출 성능이 아니라 **분류기 구성요소의 낙관적 진단값**이다.

분류기	정확도	F1 (macro)	log loss	학습 시간	파라미터
EfficientNet-B0	**0.993 ± 0.008**	**0.994 ± 0.006**	**0.166**	**2.1분**	**4.1M**
VGG16	0.993 ± 0.008	0.989 ± 0.015	0.153	5.5분	134.5M
ResNet50	0.992 ± 0.009	0.987 ± 0.015	0.217	2.1분	23.6M

표 6-2. 5-Fold 평균 ± 표준편차. 출처: reports/experiments/classifiers_f*.csv

EfficientNet-B0을 골랐다. F1이 가장 높고 VGG16의 1/33 파라미터로 2.6배 빠르다. 그러나 YOLO 박스와 결합한 2단계 모델은 fold 0에서 0.832 ~ 0.834로 YOLO 단독(0.835)을 넘지 못했다. YOLO가 이미 **학습한 약은 거의 다 맞히고 있어** 분류기가 더할 여지가 없었고, 처음 보는 약은 분류기도 학습한 적이 없기 때문이다. 분류기를 바꾸는 것으로는 open-set 문제를 풀 수 없다는 것이 이 실험의 결론이다.

6.4 효과가 없었던 시도 (음성 결과)

시도	결과	해석
Copy-Paste 합성[6] 400장	0.823 (−0.012)	이 설정만 기각. 합성이 원본(185장)의 2배를 넘어 합성 특유의 경계·배경을 학습했을 가능성. 비율을 낮춘 재실험 후보
2단계 (검출 + 분류기)	0.832 ~ 0.834	6.3절. 학습한 약은 이미 맞히고, 처음 보는 약은 분류기도 모른다
1-클래스 검출기 (위치만)	위치 0.939	56-클래스 YOLO의 위치 성능(클래스 무시 0.962)보다 낮다 → 클래스 정보가 위치 학습에도 도움
이미지당 상위 4개 제한	0.822 (−0.013)	mAP는 낮은 점수 예측까지 순위로 평가하므로, 잘라 내면 재현율을 잃는다
CLAHE 전처리	fold 2↑ 3↓	6.5절. 개선이 불확실해 미채택

표 6-3. 결과는 fold 0 mAP50-95 (CLAHE는 5-Fold). 기각한 시도도 근거와 함께 남겨 같은 실험을 반복하지 않게 했다.

6.5 전처리: CLAHE 전후 비교 (5-Fold)

EDA에서 본 Train/Test 밝기 차이를 이미지마다 국소 대비를 평탄화하는 CLAHE[5]로 줄일 수 있는지 확인했다. 알약 색은 클래스를 가르는 단서이므로 LAB의 밝기 채널에만 적용했고, 설정값은 검증 점수로 고르지 않고 OpenCV 기본값(clip 2.0, tile 8×8)으로 고정했다. 학습·검증·테스트 모두 같은 함수를 거친다.

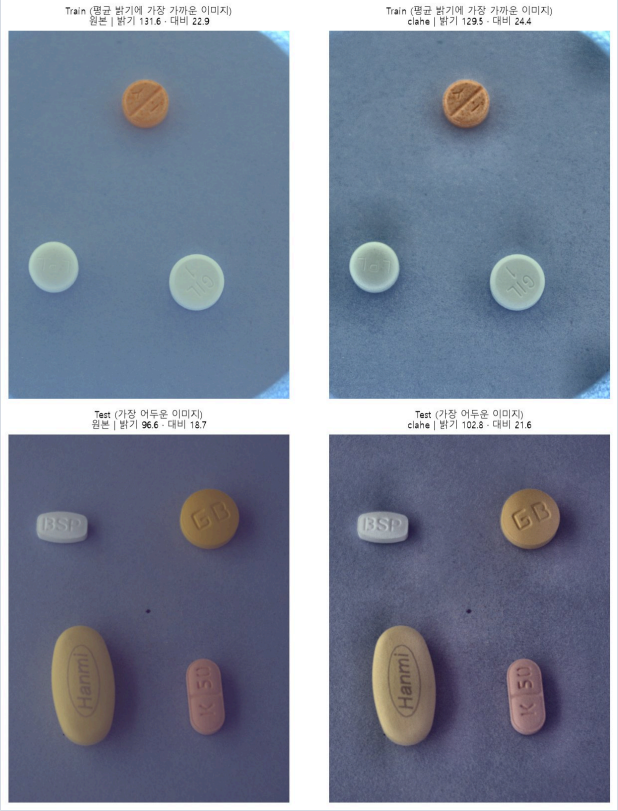


그림 6-1. CLAHE 전후. 어두운 Test 이미지가 밝아지지만 배경 질감과 알약 둘레의 어두운 테두리도 강해진다

분포 (Test − Train)	원본	CLAHE
밝기 차이	−5.55	−3.25
대비 차이	+2.88	+2.98

밝기 차이는 40% 줄었지만, 대비는 Train과 Test가 함께 커져 차이가 그대로였다.

부트스트랩 단위 – 같은 조합의 사진은 서로 독립이 아니다. 분할과 같은 단위인 조합 114개를 복원 추출(1000회)[11]해 두 실험의 차이를 짝지어 계산했다. 이미지 단위로 뽑으면 구간이 지나치게 좁아진다.

YOLO11s 5-Fold	fold 0 / 1 / 2 / 3 / 4	fold 평균	37종 조건부 OOF
원본	0.835 / 0.889 / 0.920 / 0.740 / 0.881	0.8530	0.941
CLAHE	0.832 / 0.893 / 0.921 / 0.732 / 0.880	0.8515	0.951
차이	2개 fold 상승 · 3개 fold 하락	−0.0015	+0.011 95% 구간 −0.006 ~ +0.027 (개선 확률 87%)

표 6-4. 출처: reports/experiments/preprocess_clahe_stats.csv, compare_e0_baseline_vs_e4_clahe.csv

결정: 미채택. fold 단위로는 방향이 엇갈리고, 합친 OOF의 개선도 구간이 0을 포함한다. 밝기 차이는 학습 중 HSV 증강이 이미 다루고 있어 전처리로 얻을 몫이 작았던 것으로 본다.

6.6 5-Fold 최종 후보 비교와 앙상블

모델	fold별 mAP50-95 (0 / 1 / 2 / 3 / 4)	fold 평균 ± 표준편차	OOF 56종	OOF 37종 (조건부)
YOLO11s	0.835 / 0.889 / 0.920 / 0.740 / 0.881	0.853 ± 0.070	0.621	0.941
YOLO11m	0.839 / 0.882 / 0.924 / 0.745 / 0.883	0.855 ± 0.068	0.619	0.936
WBF 11s + 11m (1:1, IoU 0.7)	–	–	**0.630**	**0.954**

표 6-5. 출처: reports/cv_summary.csv, reports/experiments/oof_analysis.csv, wbf_oof.csv

- **YOLO11s ≈ YOLO11m**: fold 평균은 11m이 +0.002, 37종 OOF는 11s가 +0.004로 방향이 엇갈린다. 단일 모델이라면 2배 빠른 11s가 낫다.
- **두 모델의 WBF[4]가 단일 모델보다 좋다**: 37종 OOF 0.941 → 0.954 (+0.013). 두 모델의 박스를 점수로 가중 평균하므로, 크기가 다른 두 모델의 위치 오차가 서로 상쇄되었을 가능성이 있다 (따로 검증하지 않은 해석).

> **설정 선택 편향 점검 (근사 메타 검증)** – WBF의 IoU 임계값과 모델 가중치 12개 조합 중 최댓값을 같은 OOF에서 고르면 낙관적이다. 그래서 fold 하나를 보류하고 나머지 4개 fold로 설정을 고른 뒤 보류 fold에만 적용하는 과정을 5번 반복했다. **5번 모두 같은 설정(1:1, IoU 0.7)**이 뽑혀 설정 선택이 안정적이었다 (보류 예측 합산 0.954).
> 다만 설정을 고를 때 쓴 다른 fold의 모델은 보류 fold를 학습에 포함했으므로 완전히 격리된 추정은 아니다. 엄밀한 nested CV는 보류 fold를 뺀 4개 fold 안에서 모델을 다시 학습해야 해, 저장된 예측만으로는 할 수 없다. 그래서 이 결과는 "설정 선택이 안정적인가"의 근거로만 쓴다.

6.7 하이퍼파라미터 · TTA

학습률 실험(R11)과 TTA 검증(R14)을 5-Fold 예측으로 진행한 뒤 결과를 기입한다.

7. 최종 후보 모델과 제출

7.1 전체 데이터 학습

최종 후보 모델은 232장 **전체**로 학습한다. 이때는 검증 데이터가 학습에 포함되므로 검증 점수로 가중치를 고를 수 없다. 검증을 끄고, 정해진 에포크만큼 학습한 **마지막 가중치(last.pt)**를 쓴다.

에포크는 fold별 최고 에포크의 **중앙값**(YOLO11s 50, YOLO11m 83)으로 정했다. 데이터가 fold 학습보다 25% 많고 학습률 스케줄이 총 에포크에 묶여 있어 최적이라는 보장은 없는 **휴리스틱**이다. 대신 fold별 학습 기록으로 민감도를 확인했다.

고정 에포크 후보	YOLO11s: 최고점 대비 평균 손해	YOLO11m: 최고점 대비 평균 손해
P25 (41 / 61)	0.010	0.009
중앙값 (50 / 83)	**0.011**	**0.007**
P75 (75 / 94)	0.004*	0.007*

표 7-1. * 조기 종료로 그 에포크까지 학습하지 않은 fold가 빠져 유리하게 치우친 값. 빠진 fold는 마지막 에포크에서도 0.003~0.012를 잃었다.

어느 후보든 손해가 약 0.01로 **최고점 주변이 평평하다.** 에포크 선택이 결과를 크게 바꾸지 않으므로 재학습 없이 중앙값을 유지했다.

7.2 제출 후보와 결과

제출 파일	구성	근거 (37종 조건부 OOF)	Kaggle public
final_11s_11m_wbf	**YOLO11s + YOLO11m WBF (1:1, IoU 0.7)**	**0.954**	**[기입]**
final_yolo11s_single	YOLO11s 단일	0.941	[기입]
final_yolo11m_single	YOLO11m 단일	0.936	[기입]

표 7-2. 테스트 842장 모두 예측 있음. 점수 0.5 이상 박스가 이미지당 3.6~3.7개로 학습 데이터(3~4개)와 일치.

Kaggle 점수를 기입하고, OOF 순위와 Kaggle 순위가 일치하는지 해석을 추가한다.

8. 결과 분석

최종 구성의 OOF 예측으로 클래스별 AP, 혼동 사례, 실패 유형(처음 보는 약 · 비슷한 약 혼동 · 박스 위치 오차)을 분석하고, 테스트 예측 예시 그림을 넣는다.

9. 재현성과 코드 품질

노트북은 분석과 해석을, 공통 코드는 `src/pilldet/` 패키지가 맡는다. 노트북과 스크립트가 같은 함수를 불러 쓰므로 결과가 어긋나지 않는다.

구성	역할
`src/pilldet/` (17개 모듈)	경로 탐색, 라벨 정제, 조합 단위 분할, YOLO/COCO 변환, 학습, 예측, COCO 평가, WBF 직접 구현, 전처리, 제출 파일 검사
`scripts/`	데이터 준비, 실험 실행(`--fold`, `--skip-train`), 5-Fold 집계, OOF 분석, 짝지은 비교, 에포크 민감도, 제출 생성
`configs/`	실험 하나 = 설정 파일 하나. 베이스라인에서 값 하나만 바꾼 파일들
`tests/`	단위 테스트 24개 (좌표 변환, 분할 누수, 평가, WBF, 제출 형식, 전처리 일치, 검사 로직)

9.1 "틀린 숫자가 조용히 나오지 않게" 하는 자동 검사

- **OOF 완전성**: fold 결과 파일 누락, 다른 fold 이미지 섞임, 예측 없는 이미지, 중복 행이 있으면 계산을 멈춘다.
- **학습 설정 스냅샷 대조**: 학습을 건너뛰고 평가만 다시 할 때 현재 설정과 학습 당시 기록(`args.yaml`)이 다르면 멈춘다. 기존 학습 20개 전부 일치 확인.
- **전처리 일치**: 학습용으로 저장한 이미지와 예측 때 실시간으로 전처리한 배열이 픽셀 단위로 같은지 테스트한다.
- **제출 파일 검사**: 컬럼 순서, 테스트에 있는 이미지 번호, 학습에 있는 약 코드, 이미지 안의 박스, 점수 범위, 중복 예측을 제출 전에 검사하고, 예측이 없는 테스트 이미지 수를 기록한다.

9.2 재현 명령

```
pip install -r requirements.txt
python scripts/prepare_data.py                      # 정제 · 변환 · 5-Fold 분할
python scripts/run_experiment.py --config configs/e0_baseline.yaml [--fold k]
python scripts/summarize_cv.py e0_baseline e2_yolo11m
python scripts/analyze_oof.py e0_baseline e2_yolo11m
python scripts/run_experiment.py --config configs/final_yolo11s.yaml   # final_yolo11m도 같은 방식
python scripts/make_submission.py --runs final_yolo11s final_yolo11m --weights 1 1 --iou-thr 0.7 --name final_11s_11m_wbf
```

GPU 학습은 cuDNN 비결정성 때문에 같은 시드에서도 가중치가 비트 단위로 같지 않을 수 있다. 재현 기준은 5-Fold 점수가 소수 셋째 자리 수준에서 같은 것이다.

10. 업무 수행 과정

날짜별 타임라인(계획 대비 실제)과 업무일지 링크, 결과 검토 반영 이력(지적 → 확인 → 조치)을 정리한다.
발표 참여: 개인 프로젝트로 전환되어 별도 발표 순서는 없고 발표일에 출석한다. 대신 시연 영상과 주강사 1:1 진행 공유 기록을 근거로 남긴다 (진행 후 기입).

11. 비즈니스 시사점과 한계

11.1 서비스에 주는 시사점

- **여러 조합에서 반복 관측된 약은 높은 진단값을 보였다.** 37종 조건부 OOF 0.954는 37종 전체를 합친 값이며, 모든 약의 정확도를 보장하지 않는다. 서비스 범위를 정하려면 약별 AP와 실패 사례(8장)를 함께 봐야 한다.
- **처음 보는 약을 다른 약으로 확신하는 것이 가장 큰 위험이다.** 베이스라인은 학습에 없던 약을 신뢰도 0.96으로 다른 약이라고 답했다. 따라서 **신뢰도 임계값만으로는 "모르는 약"을 걸러낼 수 없다.** 병용 금기 안내처럼 틀린 이름이 위험으로 이어지는 기능에는 (1) 학습에 없는 약으로 따로 만든 검증 세트에서 거부율과 임계값을 보정하고, (2) 약 이름·각인을 사용자가 확인하는 단계를 필수로 두어야 한다. 현재 데이터에는 독립된 "모르는 약" 검증 세트가 없어, 이 보고서는 운영 임계값을 제시하지 않는다.
- **모델 교체보다 데이터 확보가 먼저일 가능성이 크다.** 해상도·구조·분류기를 바꿔도 처음 보는 약 문제는 풀리지 않았다. 희귀 약의 사진을 더 모으는 것이 가장 직접적인 방법으로 보이지만, 이 프로젝트에서 효과를 측정하지는 않았다.

11.2 한계

- 한 fold에만 있는 희귀 19종의 실제 성능은 이 검증으로 잴 수 없다.
- 단색 배경·고정 촬영 환경의 데이터다. 사용자가 손으로 찍은 사진(조명, 배경, 각도)에서는 성능이 낮아질 수 있다.
- 근사 메타 검증은 완전히 격리된 추정이 아니며, 최종 에포크는 휴리스틱이다.

12. 회고와 향후 과제

프로젝트 마무리 시점에 작성한다 (잘한 점, 아쉬운 점, 다음 프로젝트에 가져갈 것).

참고문헌

1. G. Jocher, J. Qiu. *Ultralytics YOLO11*, 2024. https://github.com/ultralytics/ultralytics
2. T.-Y. Lin et al. Microsoft COCO: Common Objects in Context. *ECCV*, 2014.
3. COCO API (pycocotools). https://github.com/cocodataset/cocoapi
4. R. Solovyev, W. Wang, T. Gabruseva. Weighted boxes fusion: Ensembling boxes from different object detection models. *Image and Vision Computing* 107, 2021.
5. K. Zuiderveld. Contrast Limited Adaptive Histogram Equalization. *Graphics Gems IV*, Academic Press, 1994.
6. G. Ghiasi et al. Simple Copy-Paste is a Strong Data Augmentation Method for Instance Segmentation. *CVPR*, 2021.
7. K. Simonyan, A. Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. *ICLR*, 2015.
8. K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016.
9. M. Tan, Q. V. Le. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML*, 2019.
10. F. Pedregosa et al. Scikit-learn: Machine Learning in Python. *JMLR* 12, 2011. (StratifiedGroupKFold)
11. B. Efron, R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.
12. AI Hub. 경구약제 이미지 데이터. 한국지능정보사회진흥원. https://www.aihub.or.kr (대회 규칙에 따라 원본 조합 데이터는 사용하지 않음)